

Team members

1-Parallelization Lead: Youssuf

2-Performance Analyst & Documentation Coordinator: Rofida Khaled

3-Sequential Lead & Code Integrator / Tester: Abdelrahman Ameen

Baseline Implementation & OpenMP Parallelization

Git: <https://github.com/budi641/Parallel-Deferred-Renderer>

Performance Profiling

- 1 thread (T_S, T_P, Speedup, Efficiency):

[Baseline captured] T_S = 2638.7 ms

Full Timings:

- Total CPU time: 2638.7 ms

Performance Stats (1 threads):

- Speedup (S): 1

- Efficiency (E): 100 %

- 2 threads (T_S, T_P, Speedup, Efficiency):

Full Timings:

- Total CPU time: 2390.74 ms

Performance Stats (2 threads):

- Speedup (S): 1.10372

- Efficiency (E): 55.1858 %

- 4 threads (T_S, T_P, Speedup, Efficiency):

Full Timings:

- Total CPU time: 2067.37 ms

Performance Stats (4 threads):

- Speedup (S): 1.27636

- Efficiency (E): 31.9089 %

- 8 threads (T_S, T_P, Speedup, Efficiency):

Full Timings:

- Total CPU time: 2498.21 ms

Performance Stats (8 threads):

- Speedup (S): 1.05623

- Efficiency (E): 13.2029 %

Interpret results using Amdahl's Law & Gustafson's Law and discuss scalability limits

Amdahl's analysis

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}}$$

Best-fit $p \sim 0.167$

$1-p \sim 0.8329$

Max speedup = $0.8329/1 \sim 1.201$

Amdahl's Law predicts max ~1.20× overall speedup

Threads	Observed Total CPU (ms)	Observed Speedup	Observed Efficiency (%)	Amdahl Speedup ($p=0.1671$)	Amdahl Efficiency (%)
1	2638.70	1.00000	100.00	1.0000	100.00
2	2390.74	1.10372	55.1858	1.09117	54.56
4	2067.37	1.27636	31.9089	1.14328	28.58
8	2498.21	1.05623	13.2029	1.17125	14.64

- Amdahl shows a hard ceiling: max speedup ~1.20. This explains why adding threads gives only tiny gains and eventually no gain.
- Observed slowdown at 8 threads past the expected points to overheads.

Gustafson's analysis

For $p=0.167$

Threads (N)	Gustafson Scaled Speedup (S_G(N))
1	1.00

2	1.83
4	3.50
8	7.17

Under Gustafson's model, the renderer could in theory scale **almost linearly** if we proportionally increased the workload, like rendering more pixels, more samples, higher scene complexity).

Memory Hierarchy

- o Run cache-sensitive tests (e.g., stride or blocking).
- o Comment on cache misses / locality using perf stat or gprof.

Problem statement and baseline design

Real-time rendering in modern computer graphics involves expensive complex lighting computations. Traditional forward rendering performs lighting calculations for every object and light combination, leading to redundant and inefficient processing.

Deferred rendering separates the geometry pass (storing material and scene information) from the lighting pass (applying light computations on screen-space fragments). However, in deferred rendering, lighting and shading calculations are a bottleneck due to their high computational cost.

This project aims to improve the performance of a deferred rendering pipeline through OpenMP-based parallelization. By distributing the workload of multiple rendering passes across multiple CPU threads, we seek to achieve measurable speedup and improved efficiency without altering the visual output or rendering correctness.

Geometry Pass:

Each object's geometry is rendered once, and per-fragment data (positions, normals, and material properties) are stored into multiple render targets (the G-buffer).

Deferred Lighting Pass :

The renderer iterates over all active light sources.

For each light, it reads from the G-buffer and computes per-fragment lighting contributions using shader uniform updates and draw calls.

Forward Pass:

Visualizes point or spot lights as small meshes (spheres, cones) in the scene.

Also handled sequentially in the baseline.

Parallelization approach and OpenMP directives

The main computational bottlenecks in the deferred renderer were identified in the Lighting Pass and partially in the Forward Pass, where the program iterates over multiple light sources and performs repetitive CPU-side work (shader uniform updates, material setup, and mesh rendering).

These iterations are independent per light, presenting an opportunity for parallelization using OpenMP. The goal of this approach is to distribute the per-light operations among multiple threads to reduce total frame computation time.

- **Single Parallel Region**

A single OpenMP parallel region (`#pragma omp parallel`) is used to create a shared pool of threads for the entire lighting pass.

- **Single Thread Task Spawning**

Inside that region, the `#pragma omp single nowait` directive ensures that only one thread creates tasks while others can immediately start executing them, example: inverse view matrix, inverse projection matrix, point light updates, directional light updates).

- **Dynamic Execution**

The OpenMP runtime dynamically schedules these tasks across available threads.

Each task executes as soon as a thread becomes free, maximizing core utilization even when tasks differ in duration.

- **Synchronization**

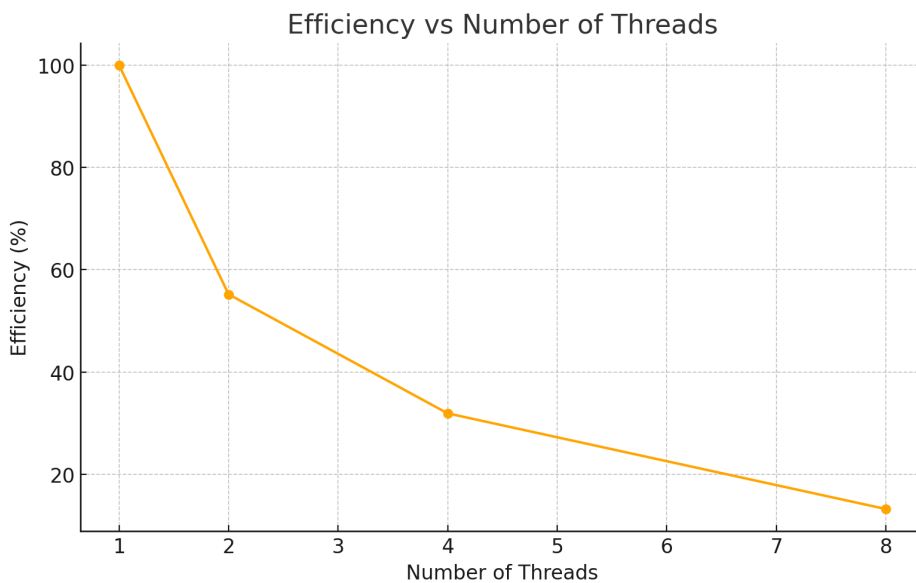
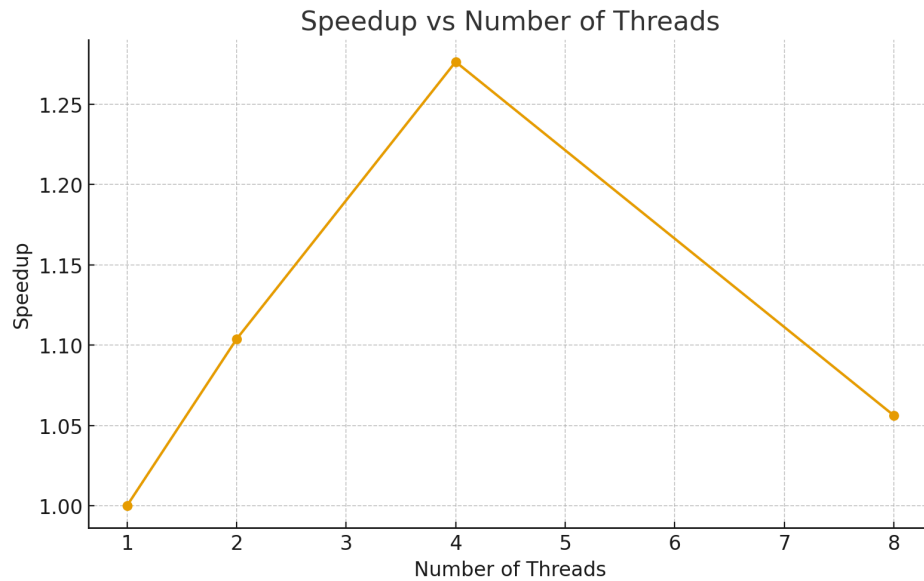
When the `#pragma omp parallel` region ends, OpenMP implicitly waits for all spawned tasks to complete before continuing, ensuring that all computed matrices and light data are ready for subsequent rendering steps.

Directives:

- `#pragma omp parallel`
- `#pragma omp single nowait`
- `#pragma omp task`
- `omp_get_wtime()`
- `omp_get_max_threads()`

Performance table + Speedup/Efficiency plots

Thread s	Total CPU Time (ms)	Speedu p	Efficiency (%)
1	2638.70	1.000	100.00
2	2390.74	1.104	55.19
4	2067.37	1.276	31.91
8	2498.21	1.056	13.20



Reflection: what bottlenecks remain?

- Around **83%** of total runtime is still serial
- Efficiency drops as threads increase, this is a sign of overhead
- Multiple threads might be competing for the same memory bandwidth shared caches.
- Certain pipeline stages like frame preparation, resource uploads, GPU command recording are inherently sequential

Demo

 WhatsApp Video 2025-11-03 at 15.43.05_96c25992.mp4